

프로젝트명	GMP 시스템 Spring Boot 3.x 전환 및 현대화 프로젝트		
연계/소속	S*운영팀 / 포커스원	개발 인원	1명 (독립 수행)
주요 업무	백엔드 개발 및 시스템 마이그레이션 전담		
담당 역할	프레임워크 마이그레이션 총괄, 멀티모듈 아키텍처 재설계, 보안 취약점 능동 발굴·수정, 인증 시스템 구축		
기술 스택	Java 17, Spring Boot 3.3.2, Spring Security 6.x, MyBatis, JPA/Hibernate, MariaDB, Gradle 8.8, Git		
업무 기간	2025.11 ~ 현재 진행중 (약 5개월)		
개발 환경	IntelliJ IDEA, Git / 운영: Linux(폐쇄망), 개발: Windows		

10년 이상 운영된 레거시 시스템 (Java 1.7 / Spring Framework 3.x)을 Java 17 / Spring Boot 3.3.2 기반으로 1인 독립 전환하며, 단순 이식이 아닌 **아키텍처 재설계 · 보안 강화 · 코드 품질 개선**을 병행했습니다.

신입임에도 프레임워크 마이그레이션 전 구간을 독립 수행하며 호환성 이슈를 직접 분류·해결했고, 폐쇄망이라는 제약 속에서 **체계적인 로깅 전략과 계층별 테스트 환경**을 직접 설계해 이슈 파악 시간을 **45% 단축**했습니다. 이후 신규 로그인 보안 시스템 설계까지 주도하며 **MyBatis + JPA 하이브리드 아키텍처**와 **Spring Security 6.x 기반 인증 구조 전환**을 완료했습니다.

53건

호환성 이슈 전량 해결

45%

이슈 파악 시간 단축

28%

빌드 시간 단축

67%

버그 수정 주기 단축

상세 내용

1. 계층별 단계 전환 전략 — 여러 계층이 한꺼번에 깨지면 원인을 찾을 수 없다

PROBLEM

Jakarta EE 전환, Spring Security 재작성, MyBatis 설정 교체, JSP 뷰 연동까지 동시에 손을 대면 어디서 오류가 났는지 추적이 불가능해집니다. 특히 코드 수정 후 폐쇄망 서버에 배포해야만 확인이 가능한 환경이었기 때문에, 원인 불명의 오류 하나가 수 시간의 낭비로 이어지는 구조였습니다.

ACTION

처음에는 문서에 나온 마이그레이션 가이드 순서대로 따라가려 했습니다. 그런데 실제로 시작해보니 여러 계층이 한꺼번에 깨지면 어디가 원인인지 알 수 없다는 걸 초반에 바로 체감했습니다.

"빌드가 되는 상태를 절대 깨지 않는다"는 원칙을 세우고, **인프라(빌드·DB 연결) → 보안·인증 → 비즈니스 로직 → 뷰** 순으로 계층을 나눠 단계적으로 진행했습니다. 디버깅의 기본인 "변수를 하나씩 제거한다"는 원리를 마이그레이션 프로세스에 그대로 적용한 것입니다.

폐쇄망 제약을 고려해 로깅 인프라를 가장 먼저 구축했습니다. ViewResolver TRACE · Spring Security DEBUG 수준의 로그 레벨을 설정하고, 클라이언트 IP와 세션ID를 로그에 자동 포함하는 **커스텀 Logback Converter**를 직접 구현했습니다. 각 단계마다 TestController와 계층별 독립 테스트 엔드포인트를 구성해 해당 계층만 검증한 뒤 다음으로 이동했습니다.

RESULT

Jakarta EE 전환부터 뷰 연동까지 전 구간을 원인 불명 오류로 인한 중단 없이 완료했습니다. 로그 기반 디버깅 환경 선구적으로 문제 발생 시 원인 파악 시간을 크게 단축했고, 버그 수정 주기도 **3일 → 1일**로 줄었습니다.

2. 데이터 접근 설계와 코드 구조 개선 — 일관성이나 효율이나

PROBLEM

신규 로그인 보안 기능을 설계하면서 세 가지 결정이 필요했습니다.
실패 이력이 여러 테이블에 분산된 구조를 어떻게 정리할지, 기존 MyBatis 외에 JPA를 도입할지,
그리고 레거시에서 이식한 코드의 필드 주입-계층 혼재 문제를 어떻게 정리할지였습니다.

ACTION

[테이블 및 인덱스 설계 최적화]

파편화되어 있던 기존 로그인 성공/실패 이력 테이블들을 **신규 단일 테이블로 구조화**하여 관리 효율을 높였습니다.
나아가 IP 차단 및 계정 잠금 로직에서 발생하는 **실제 환경의 WHERE 절 패턴을 전수 분석해 풀 테이블 스캔(Full Table Scan)**을 방지할 **복합 인덱스 7개**를 직접 설계했습니다.
특히 인덱스 카디널리티와 조건절 유형을 함께 고려해, (IP, RESULT, LOG_DT)처럼 '**동등 조건(=) → 범위 조건(<, >)**' 순서로 컬럼을 배치함으로써 무거운 이력 조회 쿼리에서도 항상 **효율적인 Index Range Scan이 보장**되도록 했습니다.

[데이터 접근 방식 선택]

처음엔 일관성을 위해 MyBatis 단일 사용을 고려했지만, 로그인 이력처럼 단순 CRUD 위주 도메인에 MyBatis XML을 새로 작성하는 건 비효율적이었습니다.
결론은 "**복잡한 동적 쿼리는 MyBatis, 신규 단순 CRUD는 JPA**"로 분리하는 것이었고, 트레이드오프 이유를 문서로 남겼습니다.
다만 두 기술을 공존시키는 과정에서 DataSource-TransactionManager 설정 충돌이 생겼고, open-in-view: false 설정과 함께 직접 검증하며 해소했습니다.

[코드 구조 개선]

필드 주입(@Autowired)을 생성자 주입으로 전환하자 빌드 시점에 CompanyService ↔ UserExService 간 순환 참조가 즉시 드러났습니다.
DI 방식이 스타일 차이가 아니라 **설계 문제를 드러내는 도구**라는 걸 직접 경험했습니다.
또한 Optional 반환 타입으로 선언한 Repository 메서드가 운영 데이터에서 2건이 조회되자 **IncorrectResultSizeDataAccessException**이 발생했습니다.
반환 타입을 List로 바꾸고 get(0)으로 최신 1건만 꺼내 쓰도록 수정해 해결했습니다.

RESULT

- 단일 테이블 통합 및 복합 인덱스 7개 직접 설계 — 조회 패턴 기반 Index Range Scan 적용
- MyBatis + JPA 하이브리드 전략으로 레거시 쿼리 자산 유지와 신규 개발 효율 양립
- 생성자 주입 전환으로 순환 참조 발굴·해소, Optional → List 전환으로 런타임 예외 해소

핵심 성과

✔ 대규모 마이그레이션 1인 완료

- Java 1.7 / Spring 3.x → Java 17 / Spring Boot 3.3.2
- Jakarta EE 전환부터 뷰 연동까지 전 구간 독립 수행 완료
- 폐쇄망 맞춤 로깅 전략 및 계층별 테스트 환경 구축

✔ 신규 보안 로그인 시스템 설계 및 구현

- IP·사번 이중 차단 정책 설계, 복합 인덱스 7개 직접 설계
- MyBatis + JPA 하이브리드, Optional → List 런타임 예외 해소
- XML 인터셉터 → SecurityFilterChain 기반 커스텀 필터로 재설계